

存算一体架构下的 Roofline 模型

从硬件分解到操作数角色分解

2026 年 9 月 7 日

摘要

经典 Roofline 模型以算力与所选存储层级的带宽描述性能上界，但统一访存量的描述不容易直接揭示存算一体 (Compute-in-Memory, CIM) 中流式求值与驻留状态更新的非对称代价。本文保留 Roofline 的需求-能力分析方式，按操作数角色区分可计算的驻留状态 (resident) 与流过该状态完成求值的输入 (streaming)。

针对指定配置和计算模式下的 CIM 求值服务单元，本文在分析工作窗口 W 内定义逻辑 streaming 输入 payload Q_S 与逻辑驻留写入 payload Q_R ，并以相同边界和口径下的服务能力 ρ 、 τ 与之配对。驻留强度 $RI = Q_S/Q_R$ 描述驻留写入的摊销关系，由此得到输入服务吞吐上界

$$P = Q_S/T \leq \min(\rho, \tau RI) \quad [\text{Byte/s}], \quad RI^* = \rho/\tau,$$

其中 T 是完成窗口工作的时间。 $Q_S > 0$ 且 $Q_R = 0$ 时取 $RI = \infty$ ，上界退化为 $P \leq \rho$ 。需求量由 workload、窗口、精度与映射共同确定；能力参数依赖声明的硬件配置和运行模式。该上界的成立不要求两种服务完全重叠，能否接近它则取决于共享资源、依赖和利用率。

本文贡献是角色化的需求-能力描述、窗口化动态驻留分析及供后续硬件-workload 匹配的计量基线。两个简短算例说明输入 Byte/s 与有用 OP/s 的换算，以及相同求值能力下驻留代价的影响。硬件数值标定、具体 workload 需求及适配结论尚待后续重建，当前长稿不将旧版数值分析作为已验证结果。

1 引言

经典 Roofline [1] 将任务的计算量和数据搬运量与平台能力配对，提供直观的性能上界。用于 CIM 时，OP/s 和访存量仍可定义，但仅给出峰值算力与统一访存带宽，不容易看出一份驻留状态需要多大代价才能建立、又能支撑多少次流式求值。本文关注这一非对称性，而非否定经典 Roofline 的数学形式。

CIM 将一类操作数保存为计算状态，另一类操作数流过该状态完成求值。权重可以驻留，也可以因容量、映射或参数更新而重写；KV 等中间状态也可以承担 resident 角色。两类操作数的角色取决于所声明的求值与映射，并不等同于固定的数据类型。

这种区分带来三个需要显式回答的问题：服务单元每次接收多少逻辑输入，完成规定输出需多少服务时间；建立或更新驻留状态需要多少逻辑写入及写入服务时间；在给定工作窗口内，同一状态的使用如何摊销写入需求。模拟 CIM (ACIM) 与数字 CIM (DCIM) 的求值路径不同，但都需要将必要的计算和读出步骤计入服务能力。

已有工作已讨论 IMC 的写入与计算受限视角，尤其 Jia 等人的分析 [24]。本文不以首次区分这两类代价或首次得到两段式曲线为贡献，而是统一两侧逻辑 payload 与能力的口径，利用窗口描述动态驻留，并为后续硬件与 workload 的配对提供可检查的输入条件。

第 2 节回顾经典 Roofline 及其计量边界。第 3 节说明操作数角色分解的动机。第 4 节给出需求、能力、窗口、上界及可达性条件，并保留对应表与两个算例。第 5 节说明后续定量重建的范围，第 6 节给出结论。

2 经典 Roofline 回顾及其计量边界

2.1 Roofline 与相关 IMC 工作

Williams、Waterman 与 Patterson 提出的 Roofline [1] 是上界与瓶颈分析工具。其 operational intensity 按所选存储层级的实际搬运量计量；缓存与局部性优化会改变该流量。原文也讨论以其他存储层级的带宽构造 Roofline。因此，单一带宽是一种选定边界的抽象，不是所有数据具有相同物理代价的声明。

与本文最接近的联系是 Jia 等人的可编程 IMC 加速器 [24]。其第 II-C 节与 Fig. 2 明确讨论 IMC-write-bound 和 IMC-compute-bound，并以每次权重写入支撑的计算量说明写入代价的摊销。需要准确区分的是，该图纵轴为能效 (OPS/W)，不能直接当作本文的吞吐图。该工作还讨论映射与复制对复用的影响。本文沿用对写入和求值非对称性的关注，以窗口内两类逻辑 payload 及对应服务能力描述吞吐上界，服务于动态驻留及后续匹配分析。

本文不从某个映射优化后的加速结果推断原 Roofline 数学失效，也不把峰值与实际性能不同视为该工具错误的证据。后文的操作数角色分解是一种补充分析抽象。

2.2 计算平台的两个关键指标

以 GPU 为例，经典 Roofline 给出两个配置相关的能力指标：

定义 2.1 (算力 π)。指定计算模式下计算路径的峰值服务能力，单位为 OP/s。精度、所用执行单元及运算计数规则必须声明。

定义 2.2 (带宽 β)。所选存储层级及传输模式下的数据搬运能力上限，单位为 Byte/s，须与任务流量使用同一层级边界。

两者分别描述计算与搬运需求的能力约束，并不意味着所有实现都能同时达到这两个峰值。

2.3 任务的计算量与搬运量

记 M 为窗口内任务的**有用 OP 总数**。本文稠密 MAC 算例统一规定 $1 \text{ MAC} = 2 \text{ OP}$ ，不将 M 同时用作 MAC 次数。对其他运算必须另行声明有用 OP 的计数规则。

GPU 执行的搬运量另记为 Q_{GPU} ，表示所选存储层级边界在该次执行中的累计传输字节，包含该边界实际发生的读写与重复搬运。它不是唯一张量大小、内存占用或空间复杂度。不同数据虽都以 Byte 计数，其访问模式、缓存行为和代价仍可不同。

2.4 算术强度、公式与几何

对 $Q_{\text{GPU}} > 0$ ，算术强度为 $\text{AI} = M/Q_{\text{GPU}}$ ，单位为 OP/Byte。它衡量每搬运一字节承载的有用计算，取决于任务、精度和执行映射，而非无条件的纯算法属性。设完成该工作的时间为 T ，则

$$T \geq \max\left(\frac{M}{\pi}, \frac{Q_{\text{GPU}}}{\beta}\right),$$

从而

$$P_{\text{OP}} = \frac{M}{T} \leq \min(\pi, \beta \text{AI}) \quad [\text{OP/s}]. \quad (1)$$

这一 $\max$ 下界仅使用两项必要服务时间；流水线重叠影响能否接近该下界，而非下界是否成立。图 1 的水平上界与斜线上界交于 $\text{AI}^* = \pi/\beta$ ，分别对应 compute-bound 与 memory-bound 分区。

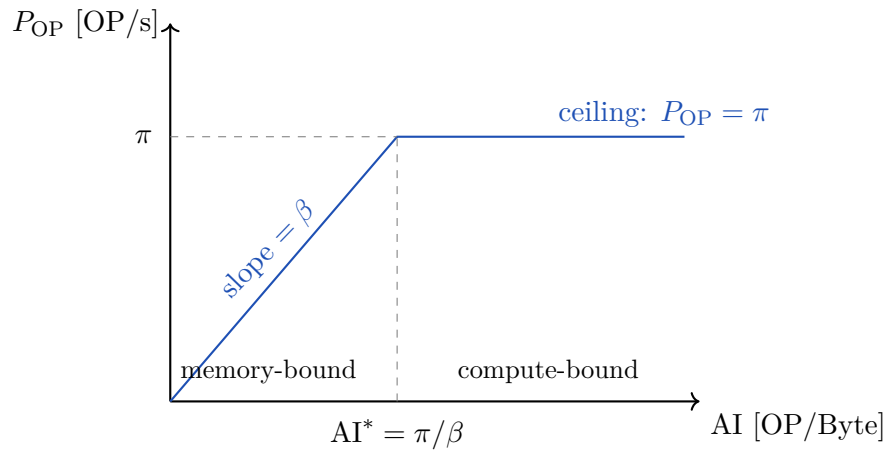


图 1: 经典 Roofline 的示意上界。曲线表示 $\min(\pi, \beta AI)$, 实际 P_{OP} 可低于曲线; 分区指出此上界中较紧的一项。图中为线性坐标示意。

2.5 使用这一抽象的边界条件

能力与需求必须使用一致的运算模式和存储层级, 重复传输必须计入需求, 共享资源与利用率会使实际性能低于上界。这些是解释结果时的计量和可达性条件。不能将其升级为关于 GPU 或 CIM 整个系统物理同质性的定理, 也不能把累计流量与经典时间/空间复杂度直接等同。

3 洞察：从硬件功能到操作数角色

3.1 CIM 求值中的非对称代价

CIM 的驻留状态建立与流式求值可具有不同的服务间隔及资源需求。对适用的电导型 ACIM，欧姆定律与基尔霍夫电流定律解释乘加的模拟实现；电荷域等其他 ACIM 实现有各自的物理求值路径。所有这些实现都需要有限的阵列响应、转换与读出时间。DCIM 则通过数字求值路径完成运算，必要的位串行展开与数字累加同样消耗服务时间。

因此，MAC 可以用 OP 计数，却不能被解释为无耗时的瞬时结果。本文把完成指定求值及输出所必需的步骤纳入 streaming 服务能力，把使驻留状态可计算所必需的步骤纳入 resident 写入能力。两类服务可共享阵列或外围资源；模型并未断言它们在物理上独立，也未断言系统只存在这两类耗时动作。

3.2 分解原则的转向

经典 Roofline 主要按计算与数据搬运的功能组织需求，本文按操作数在 CIM 求值中的角色组织需求：resident 是保存为计算状态的操作数，streaming 是输入服务中流过该状态的操作数。原稿所谓“异构维度反转”仅表达这一分析关注点的变化，不表示 GPU 数据绝对同质或 CIM 硬件绝对同质。

角色分解让建立一份状态的代价与它支撑的多次求值同时可见。物理实现仍可能包含阵列、转换器、数字逻辑、缓冲和互联，最终可达性能需要检查这些资源的共享和依赖。

3.3 以输入 Byte 描述服务工作量

本文选择**逻辑 streaming 输入 payload**作为 CIM 求值服务的工作量单位。它衡量服务单元接收并服务的输入需求，不是 GPU/HBM 流量，也不是 ADC 输出量。输出虽不直接加进 Q_S ，其必要的 sensing、ADC、累加与读出时间必须由服务能力和所声明的边界体现。

OP/s 仍可用于 CIM。输入 Byte/s 的价值是与 resident 写入 Byte/s 配对，显式呈现两类操作数服务的非对称代价。对同一次执行，可通过第 4.5 节的换算得到有用 OP/s；这不等于两个架构的实际性能相同。

4 CIM Roofline 模型

本节沿 resident-streaming 主线定义四个量，在共同边界下推导上界，并区分可达性、静态与动态工况。详细计数约定见仓库 docs/MODEL_CONVENTIONS.md，版本为 v0.1-candidate (Task 0 修订候选，待审阅)。

4.1 四个指标与驻留强度

分析对象、配置与精度 分析对象是指定配置、指定计算模式下的 CIM 求值服务单元，可以是一个 macro，或为完成声明的逻辑运算而明确组合的 macro 组。必须固定其输入接收边界、驻留写入口及输出完成边界，并声明逻辑阵列尺寸、参与的 macro/bank 数、必要的计算模式，以及 streaming 与 resident 的逻辑精度 b_S, b_R (单位均为 Byte/element)。例如 INT4 的逻辑 payload 为 0.5 Byte/element，并不表示物理总线无填充或一个物理 cell 存储一个完整权重。

Q_S 与 ρ 使用相同输入口径， Q_R 与 τ 使用相同逻辑写入口口径；四者描述同一硬件范围及兼容配置。位切片、编码、填充和位串行等实现开销由映射及服务能力体现，不能在需求与能力中重复惩罚。

分析工作窗口 W W 指明所分析的工作，而 T 是完成它的时间；二者不是同一个量。主要对象是稳态代表工作单元，也可分析 prefill 等有限阶段，但须给出窗口起止、初始驻留状态和窗口内写入。Decode 必须注明给定的上下文状态，不能将上下文持续增长的整个会话看作完全相同的无限重复周期。

一次性装载仅可在明确的预装载稳态分析中排除；冷启动或有限会话须按起始状态计入装载。每轮实际需要的重载、KV 建立或参数更新不能通过选择窗口移到分析之外。只有将同一工作模式完整重复若干次、两侧需求同比例增长，RI 才保持不变。

定义 4.1 (Streaming 输入需求 $Q_S(W)$)。窗口内由所声明服务边界接收并服务的逻辑 streaming 操作数字字节累计量，单位为 Byte。不同求值调用、分块或重放使相同输入重新进入边界时，按实际服务次数计数；同一次逻辑服务内的内部广播、位串行步骤和 ADC 多次转换不重复增加 payload。

若统计多个独立接收端口的需求，一份广播数据在各接收端分别贡献需求，这不意味着上游共享总线传输了同样多份。必须区分不同逻辑服务的重复输入和完成一次逻辑服务的内部物理步骤。

输出字节不直接加入当前 Q_S 。若输出经后处理后作为后续 CIM 求值输入重新进入服务边界，则在后一次输入服务中计数。例如 attention 的 query 输入和第二次矩阵运算的注意力系数输入分别计数；第一次矩阵运算的输出不再额外加一次。各次服务的边界和中间处理需明确，不能据此省略读出代价。 Q_S 随序列长度等参数的增长关系由 workload 和映射确定，不作通用线性假设。

定义 4.2 (Resident 写入需求 $Q_R(W)$)。窗口内为建立、更新或重新装载可计算驻留状态而由 workload / 映射发出的逻辑写入 payload 累计量，单位为 Byte。包括窗口内初始装载、实际参数更新、KV append、容量或调度触发的重载，以及同一数据的多次逻辑重写。它不是阵列容量或唯一权重大小。

同一逻辑写入事务的编程脉冲、verify 重试与内部编码进入服务代价，不按物理步骤重复增加 Q_R 。逻辑权重精度与物理 cell 位数须分开；一个 8-bit 权重不意味着单个物理 cell 存储 8 bit。

定义 4.3 (流式求值服务能力 ρ). 指定配置和模式下, 完成流式求值的输入侧逻辑 payload 服务能力上限, 单位为 Byte/s。一般表达为

$$\rho = \frac{\text{每个服务间隔完成的逻辑 streaming 输入字节}}{\text{服务间隔}}.$$

ACIM 的转换、阵列响应与读出, DCIM 的数字求值、位串行与累加等必要步骤均影响服务间隔。

单次 latency 与稳态 initiation/service interval 必须区分; 有流水线时不能默认用 latency 的倒数代替吞吐。有限窗口还可能有填充与排空代价。ADC 输出速率若用于反推 ρ , 必须结合每次逻辑服务的输入规模、转换次数及完整精度输出要求显式换算, 不能直接等同于输入服务率。

定义 4.4 (驻留写入服务能力 τ). 指定写入模式下, 使逻辑驻留 payload 进入可计算状态的服务能力上限, 单位为 Byte/s。其分母包含必要的编程、校验及适用的擦除等服务代价。小粒度更新与整页/整块更新不能无条件共用同一个有效 τ 。

ρ, τ 是上述配置与模式的能力参数, 不是脱离精度、规模和实现条件的材料固有常数。它们须来自独立的能力表征或有依据的服务模型, 不能用待预测 workload 的实际完成时间反算, 再宣称独立预测了该时间。

维护与寿命 内部 refresh、破坏性读后的 restore 等不计作新的 workload 更新; 其代价进入相应服务周期、可用能力或明确的共享资源约束。若未可靠计入, 应称为忽略该项的乐观上界, 不称为已经摊销。耐久度及目标使用寿命单列为部署可行性限制, 不混入 τ ; 评估寿命需要使用强度和写入分布等额外条件。

驻留强度及依赖关系 对 $Q_S > 0, Q_R > 0$, 定义

$$\text{RI}(W) = \frac{Q_S(W)}{Q_R(W)} \quad [\text{无量纲}]. \quad (2)$$

当 $Q_S > 0$ 且 $Q_R = 0$ 时约定 $\text{RI} = \infty$ 。下文省略 (W) 时均指给定窗口内的量。RI 衡量每字节逻辑驻留写入支撑的逻辑输入服务量; Q_S, Q_R, RI 由 workload、窗口、精度和映射共同确定。容量通过映射和重载需求改变 Q_R , 必要时也改变 Q_S 。

单 macro 的能力不能直接配整模型的唯一张量字节。跨单元汇总须说明映射、重复服务、共享资源以及能力归约方法; 不能仅将各单元的字节与速率分别相加, 就声称预测了整系统性能。理想参考映射若不被目标硬件支持, 必须重新计数后再配对。

4.2 公式与几何

公式推导 假定 $Q_S > 0, Q_R \geq 0$, 且声明模式的 $\rho > 0$ 是适用于所分析服务的能力上限。动态驻留还要求适用的 $\tau > 0$; 不支持写入的单元不能承接动态需求。完成动态窗口工作所需的两项必要服务时间给出

$$T \geq \max\left(\frac{Q_S}{\rho}, \frac{Q_R}{\tau}\right). \quad (3)$$

这一下界不要求两种服务已被证明能完全重叠。按输入服务吞吐 $P = Q_S/T$ 计量，动态分支 $Q_R > 0$ 给出

$$P = \frac{Q_S}{T} \leq \min(\rho, \tau RI) \quad [\text{Byte/s}]. \quad (4)$$

形式上仍是水平上界与斜线上界的较小值；它们不自动等于实际性能。

几何与受限分区 在 RI - P 平面（图 2），水平上界为 ρ ，斜线上界为 τRI ，交于

$$RI^* = \rho/\tau.$$

该脊点由指定配置和模式的两路能力确定。当 $RI < RI^*$ 时，resident 项更紧，称为 resident-bound；当 $RI > RI^*$ 时，streaming 项更紧，称为 streaming-bound。这是两项上界的分区，不意味着实际执行中某资源必然满载。ridge 表示 $Q_S/\rho = Q_R/\tau$ ，即两路需求时间平衡。

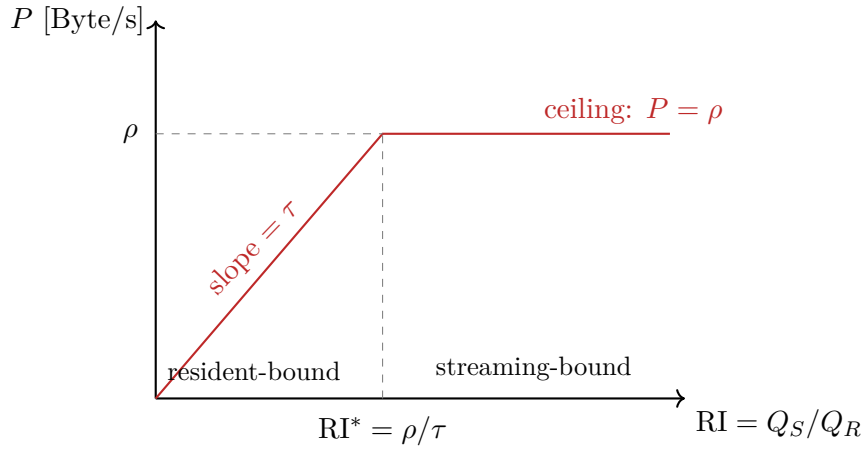


图 2: CIM Roofline 的示意上界。曲线表示 $\min(\rho, \tau RI)$ ，实际 P 可低于曲线；分区指出此上界中较紧的一项。两路需求时间在脊点平衡。图中为线性坐标示意。

静态边界 当 $Q_S > 0$ 且 $Q_R = 0$ 时，直接使用 $T \geq Q_S/\rho$ 、 $P \leq \rho$ ，无需计算任何 $M/0$ 。这只代表当前输入 Byte/s 上界不再受驻留更新项限制。任务耗时仍至少为 Q_S/ρ ，模型大小、batch、序列长度及映射仍可改变 Q_S 、利用率与所需配置，因而仍影响延迟和 token/s。

4.3 上界可达性与流水线

能否逼近式 (3)，取决于输入供给、写后读依赖、资源共享、调度和利用率。多 macro/bank 错峰可以是一种实现途径，但数量充足不是充分条件：同一资源预算下，两路单独表征的峰值未必能同时取得；外围、端口或容量共享都可能限制重叠。

对明确不能重叠、且两种服务占用同一互斥资源的简化情形，若两项分别给出该资源的必要占用时间，则更紧的约束为

$$T \geq \frac{Q_S}{\rho} + \frac{Q_R}{\tau}.$$

原 max 下界仍成立，只是更乐观。有限窗口中的流水线填充、排空和装载先于求值的依赖，也可能使实际时间增加。

两条服务能力只覆盖声明的求值服务单元。边界外的通信、非 CIM 运算和调度仍可增加任务时间，后续系统分析须明确处理。未计维护或其他必要共享开销时，必须标出忽略项，不能将

其默认视为已被掩盖。

4.4 工况、映射与优化目标

静态与动态驻留 窗口内 $Q_R = 0$ 为静态驻留； $Q_R > 0$ 为动态驻留，包括初始装载、KV 增量、实际参数更新和重载。分类是 workload 在指定窗口、驻留策略和映射下的属性，不能将所有推理无条件视为静态，也不能将某种器件永久归入一个 workload 分区。本文只计实际发生的更新，不将一次前向加权重写入的简化例子称为完整训练模型。

容量不足若触发重复装载，应将每次逻辑写入计入 Q_R ，并检查分块是否也要求输入重放。在 streaming 吞吐达到 ρ 的目标执行下，相应的 resident 需求率为

$$\frac{Q_R}{Q_S/\rho} = \frac{\rho}{\text{RI}},$$

该需求率不超过 τ 是达到该目标的必要条件，仍不保证目标可达。

优化判据 P 是输入服务吞吐，不是不同 workload 之间可直接比较的任务性能。同一任务的映射或优化应同时考察完成时间 T 、任务吞吐或有用 OP/s。减少必要重载或改善有用复用可能缓解 resident 项；提高 ρ 可能缓解 streaming 项，具体收益仍须以任务结果和资源条件评估。

不能通过增加重复输入、无用工作或降低 ρ ，使工作点更靠近 ridge 后称为性能优化。ridge 的平衡不自动表示性能最好或成本最优；当两侧逻辑精度同比例变化时，也只有服务计数与映射保持相应关系的前提下才能断言 RI 不变，能力本身还可能随精度改变。

4.5 与经典 Roofline 的对应及性能计量转换

表 1 保留两套 Roofline 的精简对应。两侧能力均须声明模式，峰值均不等于具体任务的可达性能。

维度	经典 Roofline (以 GPU 为例)	CIM Roofline
任务量	M (OP), Q_{GPU} (Byte)	Q_S, Q_R (逻辑 payload, Byte)
服务率	π (OP/s), β (Byte/s)	ρ, τ (Byte/s)
强度	$\text{AI} = M/Q_{\text{GPU}}$ (OP/Byte)	$\text{RI} = Q_S/Q_R$ (无量纲)
纵轴	$P_{\text{OP}} = M/T$ (OP/s)	$P = Q_S/T$ (输入 Byte/s)
上界	$P_{\text{OP}} \leq \min(\pi, \beta \text{AI})$	$P \leq \min(\rho, \tau \text{RI})$
左侧分区	memory-bound	resident-bound
右侧分区	compute-bound	streaming-bound
ridge	$\text{AI}^* = \pi/\beta$	$\text{RI}^* = \rho/\tau$

表 1: 两套 Roofline 的计量对应。需求量和能力均绑定各自边界与配置；分区描述上界中较紧的一项。CIM 静态分支 $Q_R = 0$ 直接取 $P \leq \rho$ 。

同一次执行的单位换算 对给定任务、窗口、精度与映射，设 $M > 0$ 为同一次执行的有效 OP 总数、 $Q_S > 0$ 。定义换算系数 $\kappa_S = M/Q_S$ ，则

$$P_{\text{OP}} = \frac{M}{T} = \kappa_S P. \quad (5)$$

对动态分支 $Q_R > 0$ ，再定义 $\kappa_R = M/Q_R$ ，于是

$$\text{RI} = \frac{\kappa_R}{\kappa_S}, \quad P_{\text{OP}} \leq \min(\kappa_S \rho, \kappa_R \tau). \quad (6)$$

κ_S, κ_R 的单位均为 OP/Byte，随任务与映射而定，不无条件称为纯算法常数。静态分支只用 $P_{\text{OP}} \leq \kappa_S \rho$ 。

上述恒等式是在同一次执行中换单位，不是 GPU 与 CIM 实际性能相等，也不产生可通用替代 GPU 参数的硬件常数。GPU 流量 Q_{GPU} 须另按其所选存储层级和实际执行计数，一般不能假设 $Q_{\text{GPU}} = Q_S + Q_R$ ，因此本文不采用通用的双向硬件等价或横轴变换。

算例一：SRAM macro 的示意单位换算 取一个逻辑权重阵列为 128×128 的 SRAM CIM 服务单元， $b_S = b_R = 1$ Byte/element (8-bit)。这是**示意配置**，不是文献实测 macro，也不声称每个物理 cell 保存一个完整 8-bit 权重。假设完整精度、满阵列 MVM 的**服务间隔**为 10 ns，已包含完成规定输出所必需的求值步骤；此值不由若干单次外围 latency 拼接，也不附加工工艺缩放结论。

权重在窗口开始前已驻留，单次 MVM 无重写。其逻辑输入为 $Q_S = 128$ Byte，MAC 次数为 $128 \times 128 = 16,384$ ，故

$$M = 2 \times 128 \times 128 = 32,768 \text{ OP}, \quad \kappa_S = 256 \text{ OP/Byte}.$$

输入服务能力及对应的满利用求值上界为

$$\rho = \frac{128 \text{ Byte}}{10 \text{ ns}} = 12.8 \text{ GB/s}, \quad (7)$$

$$P_{\text{OP}} \leq \kappa_S \rho = 3.2768 \text{ TOPS}.$$

这里 1 Byte = 8 bit，1 GB = 10^9 Byte，1 TOPS = 10^{12} OP/s；二进制容量应另用 KiB/GiB。算例只说明计量联系，不能据此断言所有 workload 都达到此吞吐；有限单次 MVM 的实际延迟也不能仅从稳态服务间隔确定。

算例二：相同求值能力，不同驻留代价 取 resident 逻辑矩阵 $n_{\text{out}} \times n_{\text{in}}$ ，一个声明的服务边界接收 U 个输入向量。参考映射假定完整矩阵可由该单元服务，各输入只进入一次，无额外重放或重载；实现所需位切片等内部步骤由服务能力体现。两种情形使用相同逻辑阵列、精度和求值能力 ρ ，完成相同的有用 MVM 工作：

$$Q_S = U n_{\text{in}} b_S, \quad M = 2 U n_{\text{out}} n_{\text{in}}. \quad (8)$$

- 矩阵在窗口开始前已驻留、窗口内不重写： $Q_R = 0$ ， $\text{RI} = \infty$ ， $P \leq \rho$ 。
- 窗口内装载一份矩阵，随后服务 U 个输入向量：

$$Q_R = n_{\text{out}} n_{\text{in}} b_R, \quad \text{RI} = \frac{U b_S}{n_{\text{out}} b_R}. \quad (9)$$

动态分支相应地有

$$\kappa_S = \frac{2 n_{\text{out}}}{b_S}, \quad \kappa_R = \frac{2 U}{b_R}, \quad P \leq \min\left(\rho, \tau \frac{U b_S}{n_{\text{out}} b_R}\right).$$

同精度方阵 GEMM 在 $U = n_{\text{in}} = n_{\text{out}}$ 时，动态分支 $\text{RI} = 1$ 。 τ 保留为符号，无需假设 SRAM

编程数值。与静态分支相比，动态分支增加了驻留服务约束，因此相同峰值求值能力不能独立决定动态任务的性能。

这里比较的是两项 Roofline 给出的吞吐上界，未假定完美重叠或实测速度。对于严格先装载完整矩阵再求值的有限窗口，依赖关系还可给出第 4.3 节所述的更紧时间约束。上述公式只适用于声明的单服务边界和参考映射，不能原封不动套用于任意多 macro 分块执行。

5 硬件与 workload 的定量标定（待重建）

本节的定量内容待后续任务按当前计量约定重建：硬件能力将给出配置、精度及服务间隔口径；workload 需求将给出窗口、驻留策略、映射及两侧 payload；适配分析将在前两者边界兼容后补入。当前未完成新的 Table I/II/III 标定或适配验证。

旧版数值分析及独立的 PD 融合、通道分解章节已保留于 `archive/task0_legacy.tex`，不参与主文编译，未按当前计量约定复核，不能作为后续表格的已验证数据直接引用。归档用于保留研究线索，不代表判定所有旧数字错误。

6 结论

本文以操作数角色分解显式描述 CIM 求值中的流式输入需求与驻留状态建立／更新需求。给定服务边界、配置、精度、窗口和映射后， Q_S, Q_R 与 ρ, τ 使用相同逻辑 payload 口径，得到 $RI = Q_S/Q_R$ 及 $P = Q_S/T \leq \min(\rho, \tau RI)$ ，脊点为 $RI^* = \rho/\tau$ 。静态驻留分支 $Q_R = 0$ 直接退化为 $P \leq \rho$ 。

该模型保留 Roofline 的简洁上界形式，同时明确驻留写入如何被输入服务摊销。上界成立与能否达到上界是两件事；重叠、共享资源、依赖、利用率和维护开销影响可达性，寿命另属部署约束。输入 Byte/s 可在同一次执行中换算为有用 OP/s，实际优化仍须关注任务时间或任务吞吐。

两个参考算例提供自治的单位换算和驻留代价比较。当前成果是供后续硬件与 workload 分析使用的候选计量基线；具体器件能力、LLM 需求及适配结论尚待按统一边界重建，本文不据旧标定作器件选型或 PD 融合的最优性判断。

参考文献

- [1] S. Williams, A. Waterman, D. Patterson. **Roofline: An Insightful Visual Performance Model for Multicore Architectures**. Communications of the ACM, vol. 52, no. 4, pp. 65–76, 2009.
- [2] Y. Zhou et al. **3D Vertical FeFET Array with Record Endurance ($> 10^{12}$), Fast Writing (± 2 V, 20 ns), Disturb Immunity, and Kb-scale Verification for High Density 1T RAM**. IEEE Symposium on VLSI Technology and Circuits, 2026.
- [3] P.-C. Wu et al. **A 28 nm 1 Mb Time-Domain Computing-in-Memory 6T-SRAM Macro with a 6.6 ns Latency, 1241 GOPS and 37.01 TOPS/W for 8b-MAC Operations for Edge-AI Devices**. IEEE International Solid-State Circuits Conference (ISSCC), 2022, pp. 1–3.
- [4] A. Guo et al. **A 22 nm 64 kb Lightning-Like Hybrid Computing-in-Memory Macro with a Compressed Adder Tree and Analog-Storage Quantizers for Transformer**

- and CNNs.** IEEE International Solid-State Circuits Conference (ISSCC), 2024, pp. 570–572.
- [5] J. Oh, C.-T. Lin, M. Seok. **D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm², 1005-Kb/mm² Digital 6T-SRAM-Based Compute-in-Memory Macro Supporting 1-to-8b Fixed-Point Arithmetic in 28 nm CMOS.** IEEE European Solid-State Circuits Conference (ESSCIRC), 2023, pp. 413–416.
 - [6] H. Fujiwara et al. **A 3 nm, 32.5 TOPS/W, 55.0 TOPS/mm² and 3.78 Mb/mm² Fully-Digital Compute-in-Memory Macro Supporting INT12×INT12 with a Parallel-MAC Architecture and Foundry 6T-SRAM Bit Cell.** IEEE International Solid-State Circuits Conference (ISSCC), 2024, pp. 572–574.
 - [7] Y.-D. Chih et al. **An 89 TOPS/W and 16.3 TOPS/mm² All-Digital SRAM-Based Full-Precision Compute-in Memory Macro in 22 nm for Machine-Learning Edge Applications.** IEEE International Solid-State Circuits Conference (ISSCC), 2021, pp. 252–254.
 - [8] W.-S. Khwa et al. **A 16 nm 96 Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3 TOPS/W and 33.2-91.2 TFLOPS/W for AI-Edge Devices.** IEEE International Solid-State Circuits Conference (ISSCC), 2024, pp. 568–570.
 - [9] W. Wan et al. **A Compute-in-Memory Chip Based on Resistive Random-Access Memory.** Nature, vol. 608, pp. 504–512, 2022.
 - [10] A. Kawahara et al. **An 8 Mb Multi-Layered Cross-Point ReRAM Macro with 443 MB/s Write Throughput.** IEEE Journal of Solid-State Circuits, vol. 48, no. 1, pp. 178–185, 2013.
 - [11] Infineon Technologies. **CY15B101N 1-Mbit (64K × 16) Automotive F-RAM Datasheet**, Document No. 001-96058, Rev. *F, 2023. Specifies $t_{CE} = 60$ ns access time, $t_{RC} = 90$ ns full read cycle, $t_{WC} = 90$ ns write cycle (NoDelay™ writes; destructive-read writeback internal to access cycle).
 - [12] S. Dutta et al. **Understanding Reliability Trade-offs in 1T-nC and 2T-nC FeRAM Designs.** IEEE Journal on Exploratory Solid-State Computational Devices and Circuits (J-XCDC), 2024. Discusses 2T-nC FeRAM enabling quasi-non-destructive readout (QNRO).
 - [13] Mythic Inc. **M1076 Analog Matrix Processor Product Brief**, v1.0, 2022. 76 AMP tiles, 80M weight parameters, 25 TOPS, 3–4 W; **GlobalFoundries 40 nm NOR-flash CIM** (process node confirmed via Mythic press release, June 2021).
 - [14] Kioxia / Samsung / SK Hynix. **3D NAND Flash datasheets and ISSCC 2024–2025 high-density papers:** 321-layer 2 Tb 4b/cell 3D-NAND Flash with 75 MB/s program throughput (ISSCC 2025); 280-layer 1 Tb with 3.2 GB/s IO rate (ISSCC 2024).
 - [15] Micron Technology. **Micron 3D TLC NAND datasheets** (96-layer family used in Micron 7400 Pro NVMe SSD, Tech. Prod. Spec., 2023): die-level $t_{PROG} \approx 800 \mu\text{s}$ typical (back-calculated from 84 MB/s die-write); $t_{BERS} \approx 15$ ms per block; page 16 KB. Older 2D 32–

- 256 Gb NAND: $t_{\text{PROG}} = 350 \mu\text{s}$ typ, $t_{\text{BERS}} = 1.5 \text{ ms}$ typ; QLC 3D NAND can rise to $t_{\text{PROG}} \sim 1.6\text{--}3 \text{ ms}$.
- [16] W. Shim, S. Yu. **System-Technology Codesign of 3-D NAND Flash-Based Compute-in-Memory Inference Engine**. IEEE Journal on Exploratory Solid-State Computational Devices and Circuits (J-XCDC), vol. 7, no. 1, pp. 60–68, 2021. Simulation-based throughput estimate; **no silicon prototype**.
 - [17] S. Liu, S. Wei, P. Yao, et al. **A 28 nm 576 K RRAM-based Compute-in-Memory Macro Featuring Hybrid Programming with Area Efficiency of 2.82 TOPS/mm²**. Journal of Semiconductors, vol. 46, no. 6, 2025, DOI:10.1088/1674-4926/24100017.
 - [18] R. Gitterman, A. Fish, A. Burg, A. Teman. **A 4-Transistor nMOS-Only Logic-Compatible Gain-Cell Embedded DRAM with Over 1.6 ms Retention Time at 700 mV in 28 nm FD-SOI**. IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 65, no. 4, pp. 1245–1256, 2018.
 - [19] B. Seyedzadeh Sany, B. Ebrahimi. **FinFET-Based Ultra-Low Power 3T GC-eDRAM with Very High Retention Time in Sub-22 nm**. Analog Integrated Circuits and Signal Processing, vol. 113, pp. 27–39, 2022. Reports 0.4 ms retention at 20 nm FinFET.
 - [20] M. Verhelst, L. Benini, N. Verma. **How to Keep Pushing ML Accelerator Performance? Know Your Rooflines!**. IEEE Journal of Solid-State Circuits, vol. 60, no. 6, pp. 1888–1905, June 2025. DOI:10.1109/JSSC.2025.3553765.
 - [21] S. Rheindt, T. Sabirov, O. Lenke, T. Wild, A. Herkersdorf. **X-Centric: A Survey on Compute-, Memory- and Application-Centric Computer Architectures**. ACM International Symposium on Memory Systems (MEMSYS), pp. 178–193, 2020.
 - [22] N. Verma, H. Jia, H. Valavi, Y. Tang, M. Ozatay, L.-Y. Chen, B. Zhang, P. Deaville. **In-Memory Computing: Advances and Prospects**. IEEE Solid-State Circuits Magazine, vol. 11, no. 3, pp. 43–55, Summer 2019.
 - [23] G. Krishnan, J. Hazra, M. Liehr, X. Du, K. Beckmann, R. V. Joshi, N. C. Cady, Y. Cao. **Design Limits of In-Memory Computing: Beyond the Crossbar**. IEEE Electron Devices Technology & Manufacturing Conference (EDTM), 2021.
 - [24] H. Jia, M. Ozatay, Y. Tang, H. Valavi, R. Pathak, J. Lee, N. Verma. **Scalable and Programmable Neural Network Inference Accelerator Based on In-Memory Computing**. IEEE Journal of Solid-State Circuits, vol. 57, no. 1, pp. 198–211, January 2022.
 - [25] A. Garofalo, G. Ottavi, F. Conti, G. Karunaratne, I. Boybat, L. Benini, D. Rossi. **A Heterogeneous In-Memory Computing Cluster for Flexible End-to-End Inference of Real-World Deep Neural Networks**. IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 12, no. 2, pp. 422–435, June 2022.
 - [26] J. Sun, P. Houshmand, M. Verhelst. **Analog or Digital In-memory Computing? Benchmarking through Quantitative Modeling**. IEEE/ACM International Conference on Computer Aided Design (ICCAD), 2023.

- [27] H. Liu, T. F. Wu, P. Houshmand, H. E. Sumbul, P. Diefenbaugh, U. Arslan, C. Tamma, B. Yi, L. Yang, M. T. Moreira, E. Beigne. **Harnessing Memory Technology Innovations for On-Device Large Language Models in Augmented Reality SoCs: Cooptimizing System Architecture with Model Characteristics.** IEEE Solid-State Circuits Magazine, Fall 2025, pp. 39–46.
- [28] T. Sharma, M. Ali, I. Chakraborty, K. Roy. **What, When, Where to Compute-in-Memory for Efficient Matrix Multiplication During Machine Learning Inference.** IEEE Transactions on Emerging Topics in Computing, vol. 13, no. 3, pp. 1215–1229, July–September 2025.
- [29] K. Zhou, J. Huang, N. S. Kim, N. Shanbhag. **A Full-System, Programmable, and Extensible In-Memory Computing Simulation Framework for Deep Learning.** 62nd ACM/IEEE Design Automation Conference (DAC), 2025.